Introdução à Computação compilando primeiro programa em C

Alexandre Rademaker

Primeiro programa em C I

src/hello.c

```c
/*
   exemplo de primeiro programa em C. O programa imprime apenas 'hello
   world' e termina.
*/

#include <stdio.h>

int main(void) {
  printf("hello, world\n");
  return 0; // retorno sem erro
}
```

Primeiro programa em C II

Comentários servem para documentar o código.

A importação do `stdio.h`. Um arquivo de cabeçalho (header) tem extensão `.h` e contém declarações de funções e definições de macros compartilhadas entre vários arquivos `.c`.

A função main é o 'entry point' em um programa em C. Ela pode receber parâmetros que permitem a comunicação entre o programa em C e o sistema operacional.

Funções recebem lista de argumentos entre parênteses, e retornam valores ou apenas produzem 'efeitos colaterais'.

Primeiro programa em C III

O nome printf é de 'formatted string'.

O ponto-e-vírgula no final de todos os comandos.

O comando `return` recebe um valor inteiro que será retornado ao SO no final da execução, zero é 'sem erro'.

A identação organiza o código em blocos lógicos e destacando a estrutura de controle do programa e tornando-o mais legível.

Vamos editar no VS Code, Replit ou GitHub Codespaces e compilar!

Primeiro programa em C IV

hello0.c — syllabus

EXPLORER

SYLLABUS

semana-1

src

agree.c
argv0.c
argv1.c
argv2.c
calculator0.c
calculator1.c
calculator2.c
calculator3.c
calculator4.c
calculator5.c
discount0.c
discount1.c
discount2.c
exit.c
hello0 U
hello0.c
hello0.s
hello1.c
mario0.c
mario1.c
mario2.c
mario3.c
mario4.c

OUTLINE

TIMELINE

hello0.c

semana-1 > src > hello0.c

```c
#include <stdio.h>

int main(void) {

  printf("hello, world\n");

}
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

zsh - src

```
ar@tenis src % make hello0
cc  -I/usr/local/include   hello0.c   -o hello0
ar@tenis src % ./hello0
hello, world
ar@tenis src %
```

main* 0 0 Live Share Ln 6, Col 1 Spaces: 2 UTF-8 LF C

Compilação I

O compilador `clang` traduz código C em um executável. Na forma mais simples:

```
> clang hello.c
> ./a.out
```

Funciona, mas sem nenhuma verificação adicional. O compilador aceita código que pode conter problemas. Para maior controle, passamos flags:

```
> clang -std=c17 -Wall -Wextra -Wpedantic -Werror
    hello.c -o hello
> ./hello
```

Em alguns ambientes, `cc` ou `gcc` ao invés de `clang`.

Compilação II

As flags do `clang` nos dão controle sobre a compilação:

- `-std=c17` versão do padrão C a ser usada
- `-Wall -Wextra -Wpedantic` ativam avisos (warnings) que ajudam a encontrar erros no código
- `-Werror` trata avisos como erros, forçando código mais rigoroso
- `-o hello` define o nome do executável de saída (sem esta flag, o padrão é `a.out`)

Compilação III

```
> clang src/cents.c
...
clang: error: linker command failed with exit code 1
    (...)

> clang src/cents.c -lm
```

Em alguns casos poderão ser necessários parâmetros adicionais para o compilador.

O programa usa a função `round` da biblioteca matemática (`math.h`). No Linux, essa biblioteca (`libm`) é separada e precisa ser vinculada explicitamente com `-lm`. No macOS, a `libm` faz parte da biblioteca do sistema (`libSystem`) e é vinculada automaticamente.

Compilação IV

O comando `make` é um utilitário que automatiza a compilação. Ele executa uma sequência de comandos (tipicamente chamadas ao compilador) seguindo regras definidas em um arquivo chamado `Makefile`.

```
> make hello
> ./hello
```

Quando **não existe** um `Makefile`, o `make` tem um comportamento default: para `make hello`, ele procura `hello.c` e chama o compilador com regras implícitas (ex: `cc hello.c -o hello`).

A limitação é justamente a falta de controle: com o `make` default, não escolhemos as flags passadas ao compilador.

Compilação V

Compilar o código na verdade significa:

- pré-processamento (pre-processing)
- compilação (compiling)
- montagem (assembling)
- vinculação (linking)

Compilação VI

src/cents.c

```
1  #include <math.h>
2  #include <stdio.h>
3
4  int main(void)
5  {
6      float amount;
7      printf("Dollar amount: ");
8      scanf("%f", &amount);
9
10     int cents = round(amount * 100);
11     printf("Cents: %i\n", cents);
12 }
```

Compilação VII

```
1  ...
2  double round(double n);
3  ...
4  int scanf(string format, ...);
5  int printf(string format, ...);
6  ...
7
8  int main(void) {
9    float amount;
10   printf("Dollar amount: ");
11   scanf("%f", &amount);
12
13   int cents = round(amount * 100);
14   printf("Cents: %i\n", cents);
15 }
```

Compilação VIII

O código C será transformado em código do processador (arquitetura) específica sendo usada.

```
> clang -S -mllvm --x86-asm-syntax=intel hello.c
```

Vide `src/hello.s`

Compilação IX

hello.a	libm.a	libc.a
01111111	0100010101	001100010001
00000010	0000000100	000001000000
00000000	0000000000	000000000000
00000000	0000000000	000000000000
00000001	0000000000	111110000000
00000001	0000000000	000100000000
00000000	0000000000	000000001000
00000000	0000000000	000000000000
00000000	0000000000	000000010000
00000000	0000000000	000000000000
10100000	0000001000	000010001000
00000000	0000000000	000000000000

Compilação X

```
hello
01111111010001010100110001000110
00000010000000010000000100000000
00000000000000000000000000000000
00000000000000000000000000000000
00000001000000000001111000000000
00000001000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
10100000000000100000000000000000
00000000000000000000000000000000
```

Erros I

Erros ocorrem, e precisamos aprender a lidar com eles!

- O que ocorre se removermos o `\n`?
- O que ocorre se quebrarmos a string?
- O que ocorre se esquecermos o ponto-e-vírgula?
- O que ocorre se removermos a biblioteca?

O símbolo `\n` e o 'escape'.

Um bom código C I

correção	seu código resolve o problema de forma correta?
design	quão eficiente é seu código ou seu algorítmo?
estilo	quão bem escrito, formatado e legível é seu código?

Um bom código C II

"Always code as if the guy who ends up maintaining your code will be a violent psychopath who knows where you live" – John Woods

"Programs must be written for people to read, and only incidentally for machines to execute." – Harold Abelson, Structure and Interpretation of Computer Programs

Mantenha seu código legível! Mais quotes!